## Exercício 1

**Item A:**

**Comandos de instalação:**
pip install opencv-python numpy

**Comandos de execução:**
python .\Ex1\ex1_itemA.py

**Evidências de execução:**

Mosaico panorâmico gerado a partir das duas imagens




Terminal:

```
(.venv) PS C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel> python .\Ex1\ex1_itemA.py
Keypoints imagem 1: 2500
Keypoints imagem 2: 2500
Matches apos Lowe: 1684
Inliers RANSAC: 1634
Erro medio de reprojecao: 0.338 pixels
Matriz de homografia:
 [[ 1.00050934e+00 -1.25772284e-04  3.29931015e+02]
 [ 1.10436945e-04  1.00015945e+00 -3.83643532e-02]
 [ 6.14380355e-07 -5.55005575e-07  1.00000000e+00]]
Mosaico salvo em: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex1\saidas\mosaico_panoramico.png
```

**Justificativa:**
Neste exercício, utilizei o ORB para encontrar pontos de interesse nas duas imagens e o BFMatcher para identificar as correspondências entre eles. Em seguida, apliquei o teste de razão de Lowe para filtrar as correspondências e o RANSAC para calcular a homografia, evitando que pontos incorretos prejudicassem o alinhamento.
Depois, utilizei o warpPerspective para alinhar as imagens e formar o mosaico panorâmico. Esse processo é parecido com o que acontece nos sistemas surround view dos veículos, que combinam imagens de várias câmeras para mostrar uma visão mais completa do que está ao redor do carro.

**Análise do resultado:**
O programa identificou 2.500 keypoints em cada imagem e encontrou 1.684 correspondências após a filtragem de Lowe. Dessas, 1.634 foram consideradas inliers pelo RANSAC, representando aproximadamente 97% das correspondências.
O erro médio de reprojeção foi de apenas 0,338 pixels, indicando que os pontos correspondentes ficaram bem alinhados.
Observando o mosaico gerado, foi possível perceber que as duas imagens foram unidas de forma consistente, sem grandes diferenças visíveis nas regiões de sobreposição.

**Como homografia multi-câmera é usada em sistemas de surround view (360°) em veículos modernos:**
Os sistemas surround view (360 graus) usam várias câmeras ao redor do carro para mostrar o que acontece em volta dele. Depois de ajustar as imagens e corrigir as distorções, as homografias ajudam a alinhar tudo em uma mesma perspectiva, como se estivéssemos olhando o carro de cima. As partes das imagens que se sobrepõem são combinadas para formar uma visão mais completa do ambiente. Porém, a homografia funciona melhor em superfícies planas. Objetos em diferentes distâncias podem ficar desalinhados por causa da paralaxe.

**Conclusão:**
Os resultados mostraram que o alinhamento foi preciso e ajudaram a entender como a homografia pode ser aplicada na combinação de imagens de diferentes câmeras em sistemas de visão veicular.

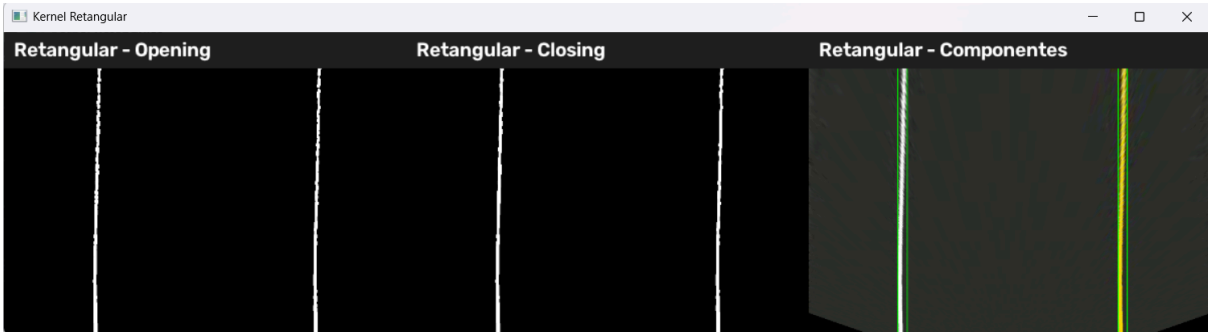
**Item B:**

**Comandos de instalação:**
pip install opencv-python numpy

**Comandos de execução:**
python .\Ex1\ex1_itemB.py

**Evidências de execução:**

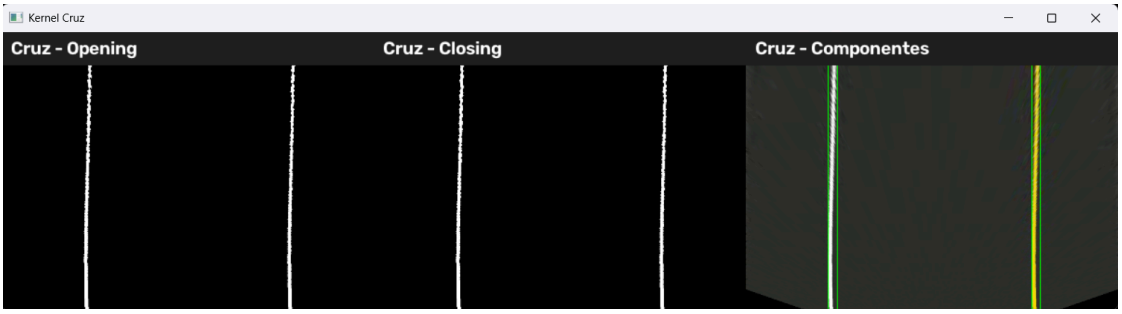
Resultado do kernel retangular



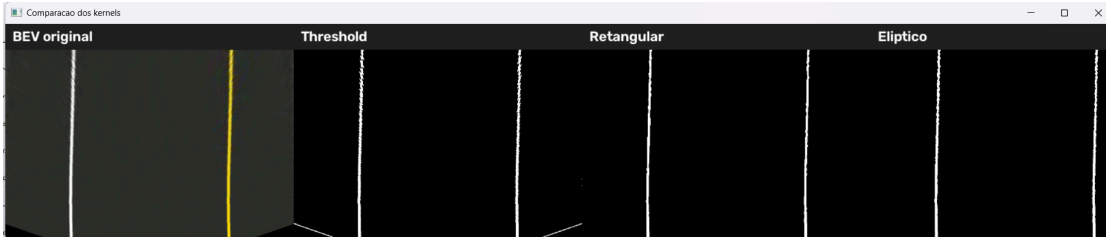

Resultado do kernel elíptico.




Resultado do kernel em cruz.




Comparação dos três kernels.




**Terminal:**

```
--- Kernel Retangular ---
Matriz do kernel:
 [[1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]]
Marcacoes detectadas: 2
Largura media: 29.00 pixels
Altura media: 720.00 pixels
Area media: 7651.00 pixels quadrados
```

```
--- Kernel Eliptico ---
Matriz do kernel:
 [[0 0 1 0 0]
 [0 1 1 1 0]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [0 1 1 1 0]
 [0 0 1 0 0]]
Marcacoes detectadas: 2
Largura media: 30.50 pixels
Altura media: 720.00 pixels
Area media: 8015.50 pixels quadrados

--- Kernel Cruz ---
Matriz do kernel:
 [[0 0 1 0 0]
 [0 0 1 0 0]
 [0 0 1 0 0]
 [1 1 1 1 1]
 [0 0 1 0 0]
 [0 0 1 0 0]
 [0 0 1 0 0]]
Marcacoes detectadas: 2
Largura media: 31.50 pixels
Altura media: 720.00 pixels
Area media: 8138.00 pixels quadrados

Resultados salvos em: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex1\saidas\itemB
```

**Justificativa:**
Neste exercício, utilizei o threshold adaptativo para separar as marcações da pista do restante da imagem BEV. Depois, apliquei as operações de opening e closing para diminuir os ruídos e corrigir pequenas falhas nas linhas.
Também testei três formatos de kernel: retangular, elíptico e cruz, para comparar os resultados e observar as diferenças entre eles.
Por último, utilizei a função cv2.connectedComponentsWithStats() para identificar as regiões detectadas e filtrar aquelas que não atendiam aos critérios de área e proporção entre altura e largura.

**Análise do resultado:**
Os três kernels conseguiram identificar duas marcações de pista, correspondentes às linhas branca e amarela presentes na imagem original. Apesar de detectarem a mesma quantidade de marcações, houve pequenas diferenças nas dimensões dos componentes. O kernel retangular apresentou a menor largura média, de 29 pixels, enquanto o kernel em cruz apresentou a maior, de 31,50 pixels. A altura média foi de 720 pixels nos três casos, mostrando que as linhas foram preservadas ao longo da imagem.
Visualmente, o opening e o closing mantiveram as marcações principais e reduziram parte dos ruídos. Os componentes conectados também foram destacados corretamente, permitindo visualizar as regiões selecionadas.

**Conclusão:**
Os resultados mostraram que todos os kernels conseguiram preservar as duas marcações principais, embora tenham produzido pequenas diferenças nas áreas e larguras detectadas.

Com isso, foi possível observar como o formato do kernel influencia a segmentação e como os componentes conectados podem ser utilizados para identificar marcações de pista em imagens BEV.

## Exercício 2

**Item A:**

**Comandos de instalação:**
pip install opencv-python numpy

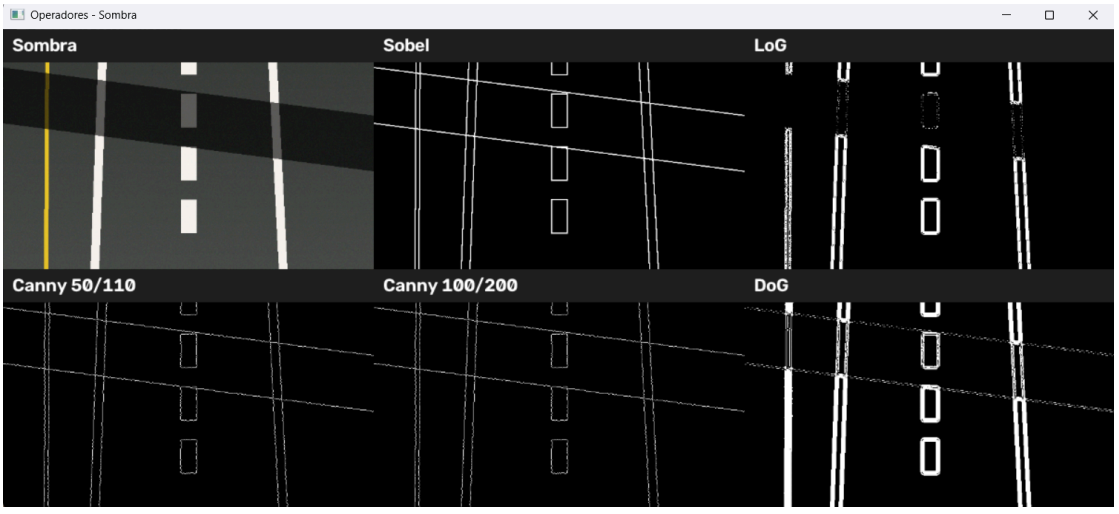
**Comandos de execução:**
python .\Ex2\ex2_itemA.py

**Evidências de execução:**

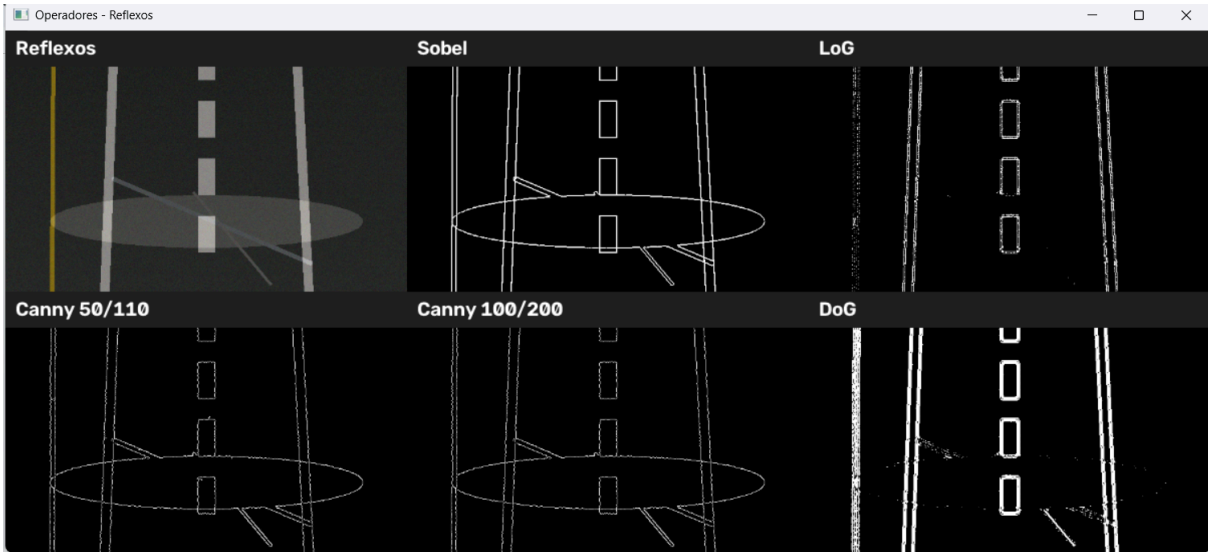
Comparação dos operadores na condição normal.




Comparação dos operadores na condição de sombra.



Comparação dos operadores na condição de reflexos.




**Terminal:**

```
(.venv) PS C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel> python .\Ex2\ex2_itemA.py

COMPARACAO DE OPERADORES DE BORDA
--------------------------------------------------------------------------
Cenario      Operador            Pixels ativos     Tempo (ms)
--------------------------------------------------------------------------
Normal       Sobel                        6280          2.935
Normal       LoG                         16227          1.674
Normal       Canny 50/110                 3048          0.653
Normal       Canny 100/200                3048          0.439
Normal       DoG                         22954          1.208
Sombra       Sobel                        8758          3.575
Sombra       LoG                         12978          2.214
Sombra       Canny 50/110                 4300          0.989
Sombra       Canny 100/200                4300          0.489
Sombra       DoG                         21629          0.463
Reflexos     Sobel                        8741          3.079
Reflexos     LoG                          6367          1.902
Reflexos     Canny 50/110                 4559          0.704
Reflexos     Canny 100/200                4506          0.404
Reflexos     DoG                         18041          0.460

Tabela salva em: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex2\saidas\itemA\comparacao_operadores.csv

CONCLUSAO:
Compare a continuidade das faixas e as bordas falsas.
Canny e um candidato, mas a melhor configuracao
deve ser escolhida apos analisar as imagens geradas.
Sombra e reflexos utilizados sao simulados.
(.venv) PS C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel>
```

**Justificativa:**

Neste exercício, utilizei quatro operadores de detecção de bordas: Sobel, LoG, Canny e DoG. No caso do Canny, testei dois pares diferentes de thresholds para comparar os resultados. A ideia foi observar como cada operador identifica as bordas das faixas da pista e como eles se comportam quando a iluminação muda.

Para fazer essa comparação, utilizei uma imagem em condição normal e criei outras duas versões, uma com sombra e outra com reflexos simulados. Também contei a quantidade de pixels ativados e medi o tempo de processamento de cada operador, para conseguir comparar tanto os resultados nas imagens quanto os valores obtidos nos testes.

**Análise do resultado:**
Na imagem em condição normal, todos os operadores conseguiram identificar as marcações da pista, mas houve algumas diferenças na espessura e na quantidade de bordas encontradas. O Sobel conseguiu manter os contornos das faixas, enquanto o LoG e o DoG destacaram mais algumas regiões da imagem. O DoG foi o que apresentou a maior quantidade de pixels ativados nos três cenários, mas isso não quer dizer que ele tenha identificado melhor as faixas. Na imagem com sombra, percebi que alguns operadores também detectaram as bordas da região escurecida, mesmo não sendo marcações da pista. Algo parecido aconteceu na imagem com reflexos, em que apareceram bordas de outras partes da cena. Já o Canny apresentou bordas mais finas e conseguiu manter boa parte das marcações visíveis nas três condições. Os dois pares de thresholds tiveram resultados muito parecidos, mas o Canny 100/200 foi o mais rápido nos três testes, chegando a 0,404 ms na imagem com reflexos.

**Conclusão:**
Considerando os resultados visuais e os tempos obtidos, o Canny 100/200 apresentou o melhor equilíbrio entre preservação dos contornos das faixas e desempenho de processamento nos cenários testados.
Apesar disso, ele também identificou algumas bordas causadas pelas sombras e pelos reflexos. Portanto, os resultados indicam que o Canny foi a opção mais adequada neste experimento, mas seriam necessários testes com imagens reais de asfalto molhado e sombra para confirmar sua robustez em situações de trânsito.

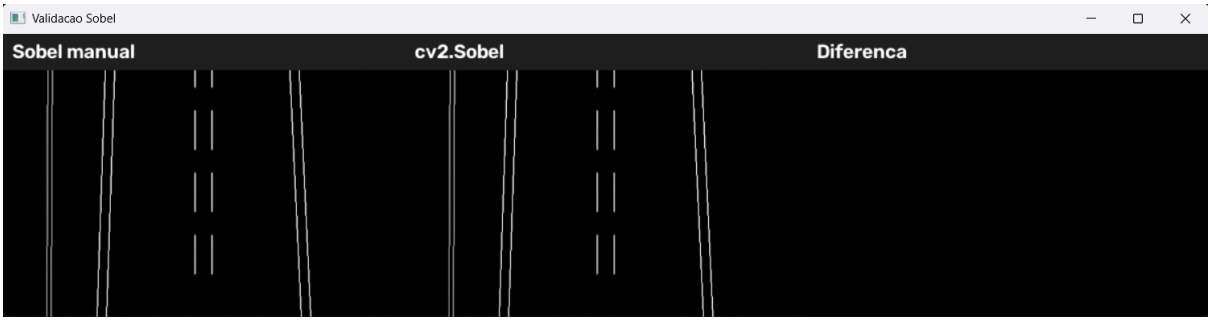
**Item B:**

**Comandos de instalação:**
pip install opencv-python numpy

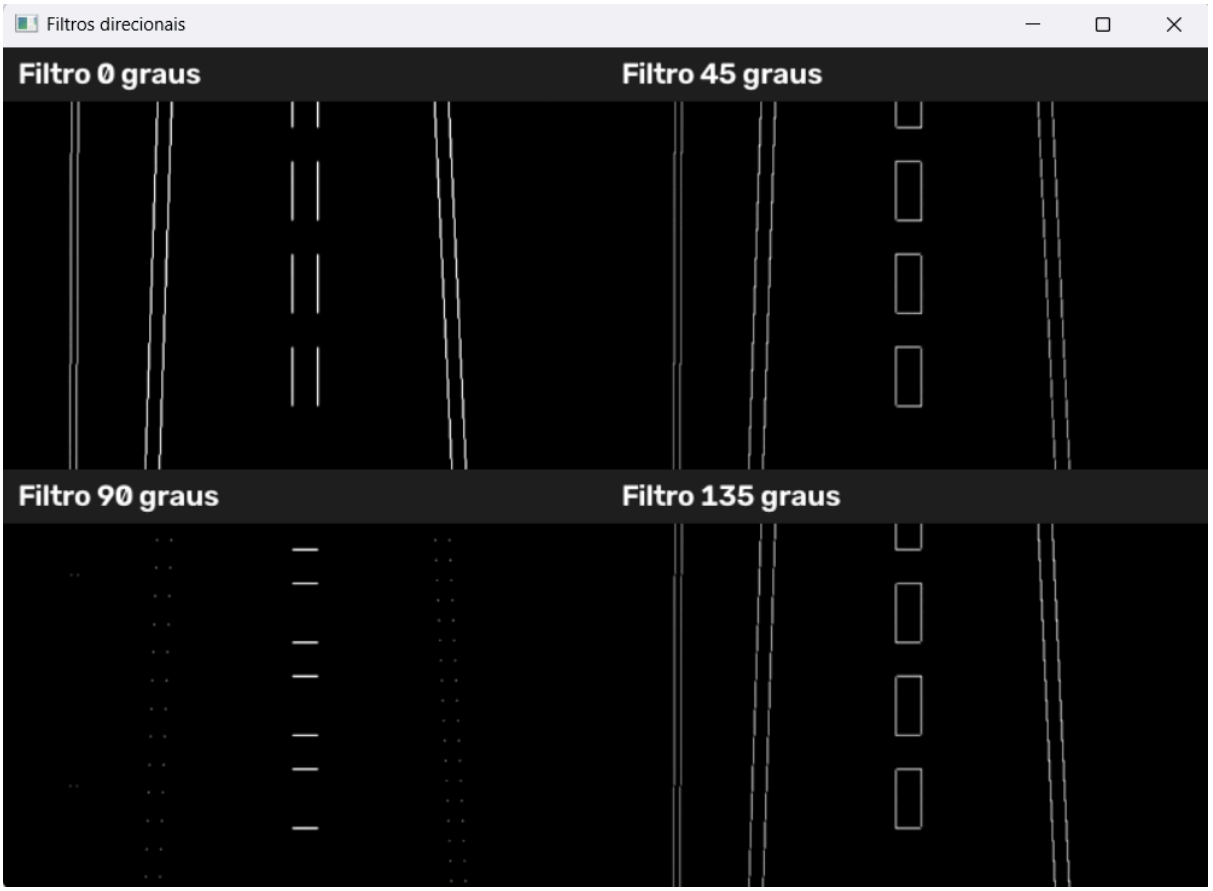
**Comandos de execução:**
python .\Ex2\ex2_itemB.py

**Evidências de execução:**

Comparação entre a convolução manual e o Sobel do OpenCV.



Respostas dos quatro filtros direcionais.




**Terminal:**

```
(.venv) PS C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel> python .\Ex2\ex2_itemB.py

VALIDACAO SOBEL MANUAL X OPENCV
Erro maximo: 0.0
Erro medio: 0.0
Pixels diferentes: 0
Total de pixels: 256000
Comparacao exata: True

BANCO DE FILTROS DIRECIONAIS
----------------------------------------
Angulo   0 graus: 588.467
Angulo  45 graus: 447.254
Angulo  90 graus: 68.002
Angulo 135 graus: 450.212

Angulo de maior resposta: 0 graus
Resposta media maxima: 588.467

RELACAO COM CNN
Entrada H x W x 1: (400, 640, 1)
Saida H x W x 4: (400, 640, 4)

Resultados salvos em: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex2\saidas\itemB
```

**Justificativa:**
Neste exercício, fiz uma convolução 2D manual usando NumPy e um kernel Sobel 3×3, sem utilizar a função cv2.filter2D. Depois, comparei o resultado com o cv2.Sobel para conferir se os valores encontrados eram iguais em todos os pixels.
Também criei quatro filtros direcionais, com ângulos de 0°, 45°, 90° e 135°, e apliquei cada um na imagem BEV da pista. A ideia foi comparar como eles identificam as bordas das marcações e descobrir qual direção apresenta a resposta mais forte.
Essa atividade também tem relação com o que foi estudado na disciplina anterior, já que as CNNs utilizam filtros para identificar características nas imagens. A principal diferença é que, neste exercício, defini os filtros manualmente, enquanto nas CNNs os valores desses filtros são aprendidos durante o treinamento.

**Análise do resultado:**
Na primeira parte, a convolução manual apresentou exatamente o mesmo resultado do cv2.Sobel. Tanto o erro máximo quanto o erro médio foram iguais a zero, ou seja, não houve diferença em nenhum dos 256.000 pixels analisados. Isso também apareceu na comparação das imagens, já que a parte que mostrava as diferenças ficou completamente preta. Na segunda parte, percebi que os filtros direcionais tiveram resultados diferentes dependendo da direção das bordas. O filtro de 0° apresentou a maior resposta média, com 588,467, seguido pelos filtros de 135° e 45°. Já o filtro de 90° teve a menor resposta, com 68,002. Isso faz sentido porque as marcações da pista na imagem são principalmente verticais. Como o filtro de 0° identifica mudanças de intensidade na direção horizontal, ele conseguiu destacar melhor as laterais das faixas. Já o filtro de 90° destacou mais as extremidades horizontais das linhas tracejadas.
Por último, os resultados dos quatro filtros foram reunidos em uma estrutura de dimensões (400, 640, 4), formando quatro canais de características. Isso ajuda a entender como uma CNN consegue aplicar diferentes filtros em uma mesma imagem e gerar vários mapas de características.

**Conclusão:**
O exercício mostrou que é possível implementar uma convolução 2D manual e obter o mesmo resultado da função Sobel do OpenCV, comprovado pela comparação pixel a pixel sem nenhuma diferença.
Na análise dos filtros direcionais, o ângulo de 0° apresentou a maior resposta sobre as marcações da pista, principalmente devido à orientação vertical das faixas na imagem BEV.
Também foi possível relacionar o funcionamento desses filtros às CNNs estudadas em DR2, entendendo como diferentes filtros podem extrair características de uma imagem e gerar múltiplos mapas de resposta.

## Exercício 3

**Item A:**

**Comandos de instalação:**
pip install opencv-python numpy

**Comandos de execução:**

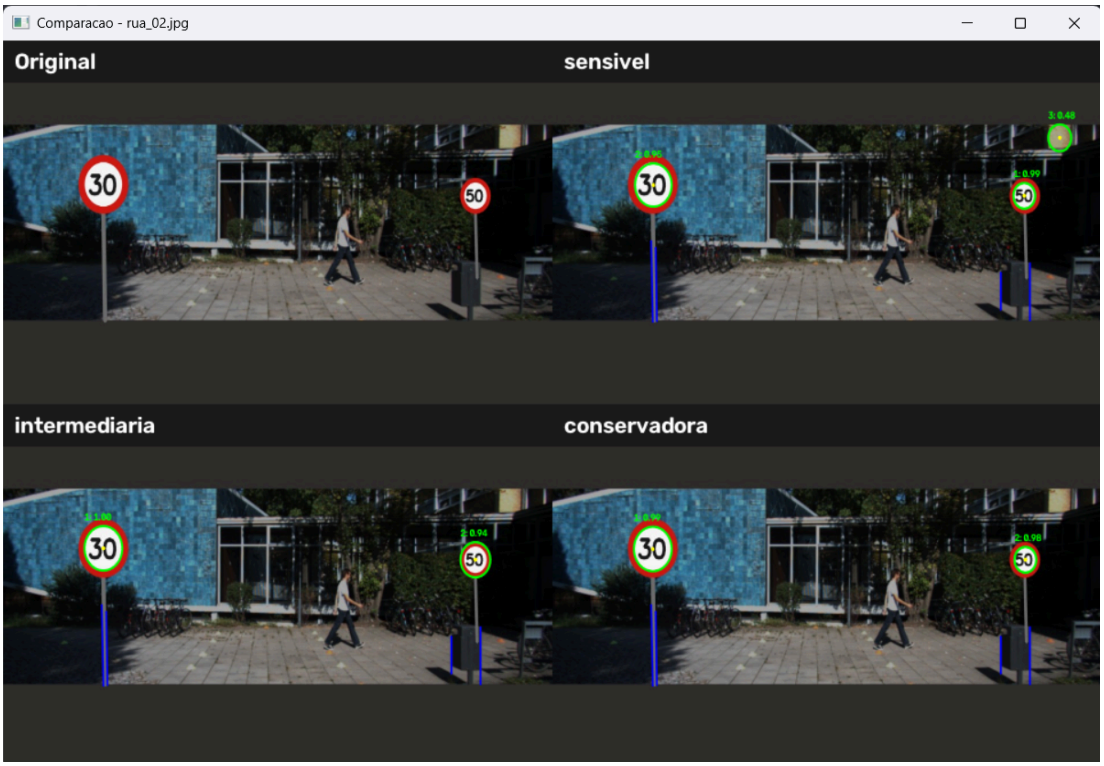
python .\Ex3\ex3_itemA.py

**Evidências de execução**

Detecção na imagem rua_01.jpg.



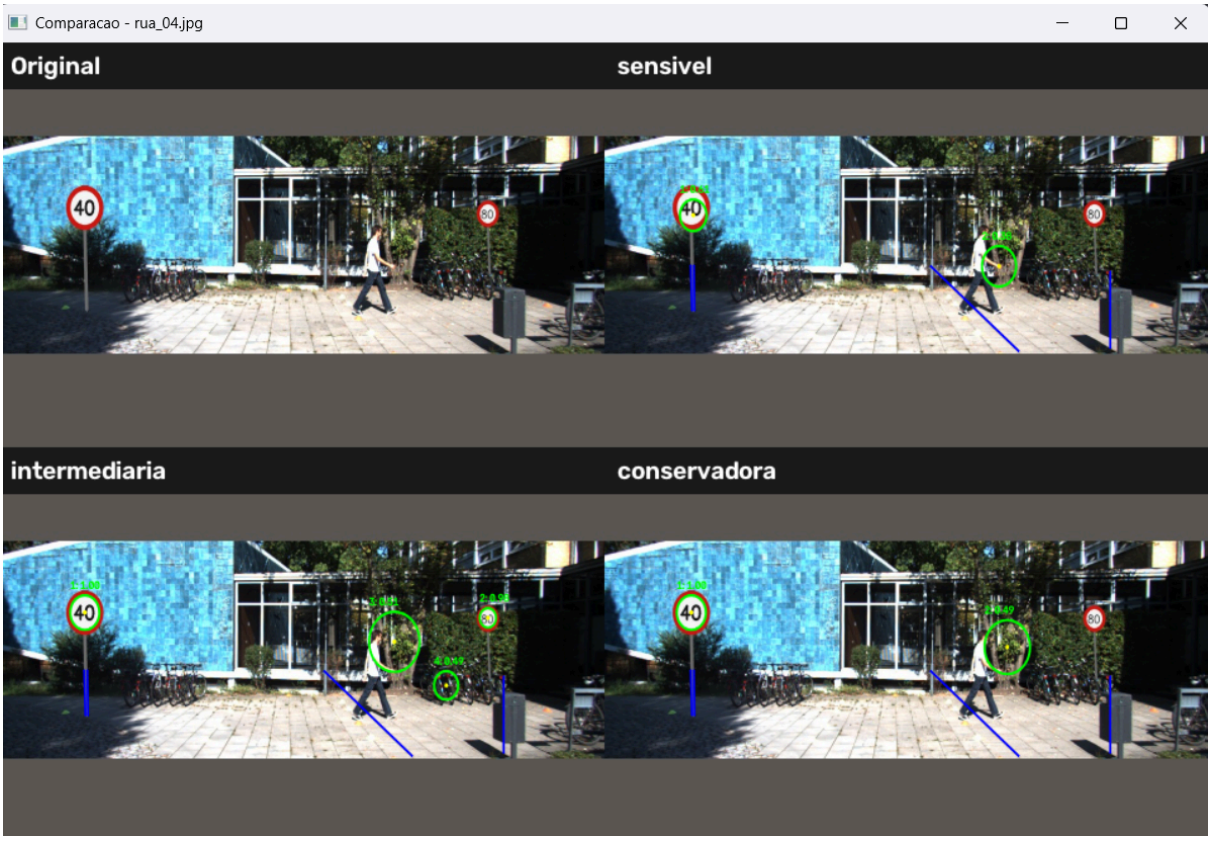

Detecção na imagem rua_02.jpg.



Detecção na imagem rua_03.jpg.




Detecção na imagem rua_04.jpg.



Detecção na imagem rua_05.jpg.




**Terminal:**

```
IMAGEM: rua_01.jpg
Linhas detectadas: 10

Configuracao: sensivel
dp=1.0, minDist=25, param1=100, param2=20
Circulos candidatos: 24
Circulos apos filtragem: 1
  Circulo 1: centro=(820,206), raio=39, confianca_estimada=0.975, suporte_borda=0.950, suporte_vermelho=1.000

Configuracao: intermediaria
dp=1.2, minDist=35, param1=135, param2=32
Circulos candidatos: 1
Circulos apos filtragem: 1
  Circulo 1: centro=(820,205), raio=40, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000

Configuracao: conservadora
dp=1.5, minDist=50, param1=160, param2=45
Circulos candidatos: 1
Circulos apos filtragem: 1
  Circulo 1: centro=(820,205), raio=40, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000

IMAGEM: rua_02.jpg
Linhas detectadas: 4

Configuracao: sensivel
dp=1.0, minDist=25, param1=100, param2=20
Circulos candidatos: 108
Circulos apos filtragem: 3
  Circulo 1: centro=(824,230), raio=19, confianca_estimada=0.989, suporte_borda=0.978, suporte_vermelho=1.000
  Circulo 2: centro=(176,214), raio=32, confianca_estimada=0.947, suporte_borda=0.894, suporte_vermelho=1.000
  Circulo 3: centro=(886,144), raio=20, confianca_estimada=0.481, suporte_borda=0.594, suporte_vermelho=0.367
```

```
  Configuracao: intermediaria
  dp=1.2, minDist=35, param1=135, param2=32
  Circulos candidatos: 21
  Circulos apos filtragem: 2
    Circulo 1: centro=(175,213), raio=33, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000
    Circulo 2: centro=(825,230), raio=26, confianca_estimada=0.936, suporte_borda=0.872, suporte_vermelho=1.000

  Configuracao: conservadora
  dp=1.5, minDist=50, param1=160, param2=45
  Circulos candidatos: 8
  Circulos apos filtragem: 2
    Circulo 1: centro=(175,214), raio=33, confianca_estimada=0.994, suporte_borda=0.989, suporte_vermelho=1.000
    Circulo 2: centro=(826,230), raio=19, confianca_estimada=0.978, suporte_borda=0.956, suporte_vermelho=1.000

  IMAGEM: rua_03.jpg
  Linhas detectadas: 12

  Configuracao: sensivel
  dp=1.0, minDist=25, param1=100, param2=20
  Circulos candidatos: 70
  Circulos apos filtragem: 7
    Circulo 1: centro=(114,164), raio=47, confianca_estimada=0.947, suporte_borda=0.894, suporte_vermelho=1.000
    Circulo 2: centro=(150,166), raio=29, confianca_estimada=0.517, suporte_borda=0.517, suporte_vermelho=0.517
    Circulo 3: centro=(72,158), raio=20, confianca_estimada=0.467, suporte_borda=0.378, suporte_vermelho=0.556
    Circulo 4: centro=(124,190), raio=37, confianca_estimada=0.456, suporte_borda=0.406, suporte_vermelho=0.506
    Circulo 5: centro=(96,146), raio=36, confianca_estimada=0.456, suporte_borda=0.400, suporte_vermelho=0.511
    Circulo 6: centro=(126,138), raio=33, confianca_estimada=0.439, suporte_borda=0.361, suporte_vermelho=0.517
    Circulo 7: centro=(94,182), raio=22, confianca_estimada=0.411, suporte_borda=0.494, suporte_vermelho=0.328

  Configuracao: intermediaria
  dp=1.2, minDist=35, param1=135, param2=32
  Circulos candidatos: 4
  Circulos apos filtragem: 1
    Circulo 1: centro=(115,165), raio=48, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000
```

```
Configuracao: conservadora
dp=1.5, minDist=50, param1=160, param2=45
Circulos candidatos: 1
Circulos apos filtragem: 1
  Circulo 1: centro=(115,164), raio=48, confianca_estimada=0.972, suporte_borda=0.944, suporte_vermelho=1.000

IMAGEM: rua_04.jpg
Linhas detectadas: 4

Configuracao: sensivel
dp=1.0, minDist=25, param1=100, param2=20
Circulos candidatos: 259
Circulos apos filtragem: 2
  Circulo 1: centro=(140,230), raio=21, confianca_estimada=0.608, suporte_borda=0.606, suporte_vermelho=0.611
  Circulo 2: centro=(624,298), raio=27, confianca_estimada=0.375, suporte_borda=0.444, suporte_vermelho=0.306

Configuracao: intermediaria
dp=1.2, minDist=35, param1=135, param2=32
Circulos candidatos: 99
Circulos apos filtragem: 4
  Circulo 1: centro=(137,220), raio=23, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000
  Circulo 2: centro=(775,227), raio=13, confianca_estimada=0.978, suporte_borda=0.956, suporte_vermelho=1.000
  Circulo 3: centro=(627,259), raio=40, confianca_estimada=0.508, suporte_borda=0.711, suporte_vermelho=0.306
  Circulo 4: centro=(709,317), raio=19, confianca_estimada=0.494, suporte_borda=0.500, suporte_vermelho=0.489

Configuracao: conservadora
dp=1.5, minDist=50, param1=160, param2=45
Circulos candidatos: 37
Circulos apos filtragem: 2
  Circulo 1: centro=(137,220), raio=23, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000
  Circulo 2: centro=(637,266), raio=36, confianca_estimada=0.486, suporte_borda=0.667, suporte_vermelho=0.306

IMAGEM: rua_05.jpg
Linhas detectadas: 10
```

```
Configuracao: sensivel
dp=1.0, minDist=25, param1=100, param2=20
Circulos candidatos: 7
Circulos apos filtragem: 2
  Circulo 1: centro=(134,218), raio=15, confianca_estimada=0.994, suporte_borda=0.989, suporte_vermelho=1.000
  Circulo 2: centro=(786,190), raio=27, confianca_estimada=0.983, suporte_borda=0.967, suporte_vermelho=1.000

Configuracao: intermediaria
dp=1.2, minDist=35, param1=135, param2=32
Circulos candidatos: 2
Circulos apos filtragem: 2
  Circulo 1: centro=(785,190), raio=28, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000
  Circulo 2: centro=(135,218), raio=21, confianca_estimada=0.972, suporte_borda=0.944, suporte_vermelho=1.000

Configuracao: conservadora
dp=1.5, minDist=50, param1=160, param2=45
Circulos candidatos: 2
Circulos apos filtragem: 2
  Circulo 1: centro=(785,190), raio=28, confianca_estimada=1.000, suporte_borda=1.000, suporte_vermelho=1.000
  Circulo 2: centro=(134,218), raio=21, confianca_estimada=0.961, suporte_borda=0.922, suporte_vermelho=1.000

RESUMO FINAL
Imagens processadas: 5
Configuracoes por imagem: 3
Circulos candidatos: 644
Circulos apos filtragem: 33
Resultados salvos em: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex3\saidas\itemA
(.venv) PS C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel>
```

**Justificativa:**
Neste exercício, utilizei a transformada de Hough para identificar sinais de trânsito circulares e marcações de faixa em imagens reais de ruas.
Primeiro, converti as imagens para tons de cinza e apliquei o Gaussian Blur para suavizá-las. Depois, utilizei o cv2.HoughCircles com três configurações diferentes para comparar a quantidade de círculos encontrados e verificar quais resultados estavam mais próximos dos sinais presentes nas imagens.
Também apliquei uma filtragem usando as bordas e a cor vermelha dos sinais para tentar diminuir as detecções incorretas. Para calcular a confiança de cada detecção, considerei a presença de bordas e da cor vermelha ao redor do círculo. Esse valor serviu como uma estimativa para avaliar as detecções, e não como uma probabilidade exata de acerto.
Por último, utilizei o cv2.HoughLinesP para identificar linhas na parte inferior das imagens, onde ficam as marcações da pista. Para facilitar a visualização, desenhei as linhas detectadas em azul e os círculos em verde.

**Análise do resultado:**
Os resultados mostraram que os parâmetros escolhidos no HoughCircles fazem bastante diferença na quantidade de círculos encontrados e na qualidade das detecções.
A configuração sensível encontrou mais círculos, mas também identificou várias regiões que não eram sinais, principalmente nas imagens rua_03 e rua_04. Já a configuração conservadora conseguiu diminuir essas detecções incorretas, mas acabou não identificando o sinal menor de 80 km/h na rua_04.
A configuração intermediária foi a que apresentou os melhores resultados, conseguindo identificar os sinais nas cinco imagens sem gerar tantos falsos positivos. Ela encontrou os dois sinais da rua_02, os dois da rua_04 e também os sinais que estavam em diferentes distâncias na rua_05.

A filtragem por cor e bordas também ajudou bastante, diminuindo de 644 círculos candidatos para 33 detecções. Mesmo assim, algumas partes da vegetação e outros objetos das imagens ainda foram confundidos com sinais circulares.
Na parte de detecção de linhas, o HoughLinesP conseguiu destacar algumas marcações da pista, principalmente nas imagens de rodovia. Porém, também detectou linhas que não faziam parte das faixas, como postes e outras estruturas presentes nas ruas.

**Conclusão:**
A comparação mostrou que a escolha dos parâmetros influencia bastante os resultados, sendo a configuração intermediária a mais equilibrada entre as três testadas. A filtragem por cor vermelha e suporte de bordas também ajudou a diminuir significativamente os falsos positivos. Apesar dos bons resultados na identificação dos sinais, ainda ocorreram algumas detecções incorretas, principalmente em cenários com muitos elementos visuais. Isso mostra que as técnicas são úteis para a percepção veicular, mas precisam de cuidados adicionais para funcionar de maneira confiável em condições reais de trânsito.

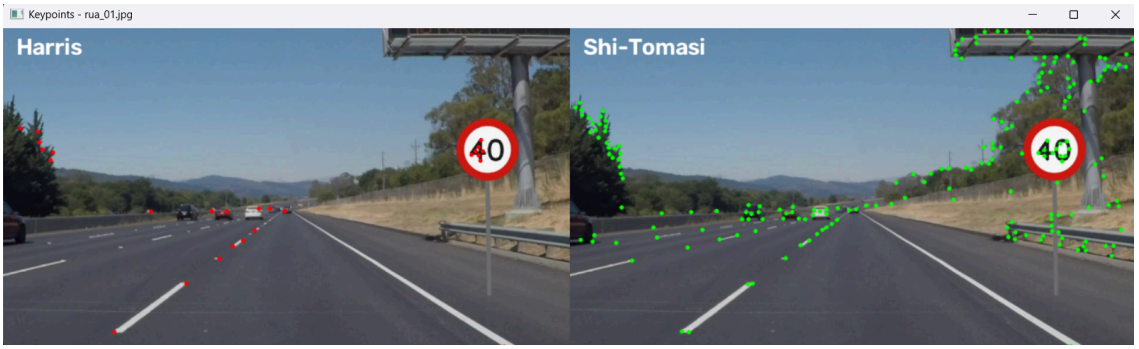
**Item B:**

**Comandos de instalação:**
pip install opencv-python numpy

**Comandos de execução:**
python .\Ex3\ex3_itemB.py

**Evidências de execução:**

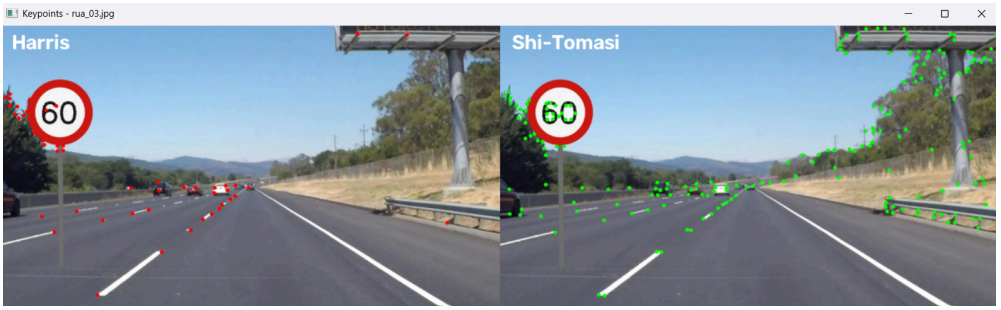
Comparação entre Harris e Shi-Tomasi na imagem rua_01.jpg.




Comparação entre Harris e Shi-Tomasi na imagem rua_02.jpg.



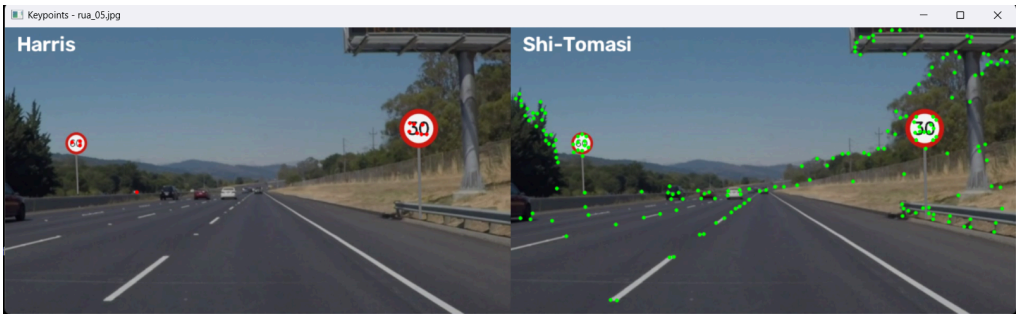
Comparação entre Harris e Shi-Tomasi na imagem rua_03.jpg.




Comparação entre Harris e Shi-Tomasi na imagem rua_04.jpg.




Comparação entre Harris e Shi-Tomasi na imagem rua_05.jpg.

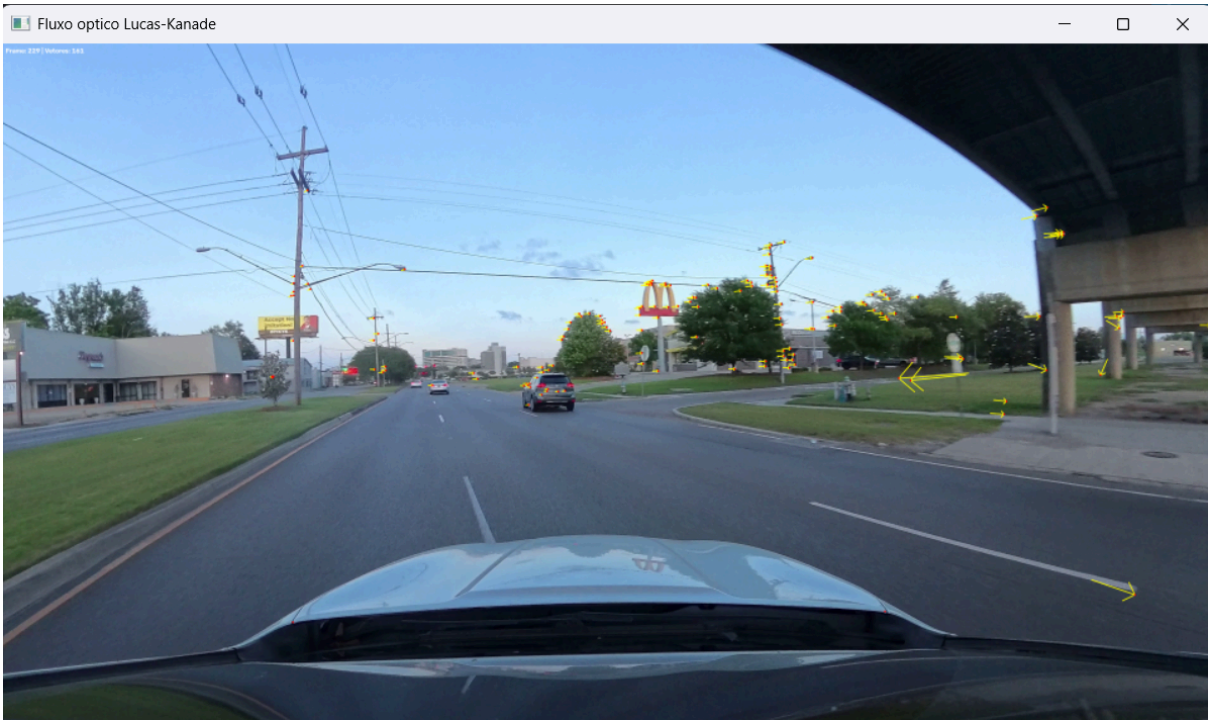



**prints do vídeo**

Fluxo óptico Lucas-Kanade durante o deslocamento do veículo.



Fluxo óptico Lucas-Kanade próximo ao cruzamento.




**Terminal:**

```
COMPARACAO HARRIS X SHI-TOMASI
----------------------------------------------------------------

Imagem: rua_01.jpg
Harris: 27 pontos | 8.349 ms
Shi-Tomasi: 200 pontos | 8.032 ms

Imagem: rua_02.jpg
Harris: 89 pontos | 10.121 ms
Shi-Tomasi: 200 pontos | 9.081 ms

Imagem: rua_03.jpg
Harris: 51 pontos | 10.261 ms
Shi-Tomasi: 200 pontos | 6.377 ms

Imagem: rua_04.jpg
Harris: 200 pontos | 13.923 ms
Shi-Tomasi: 200 pontos | 8.916 ms

Imagem: rua_05.jpg
Harris: 11 pontos | 20.284 ms
Shi-Tomasi: 182 pontos | 7.263 ms

MEDIAS DOS TEMPOS
Harris: 12.588 ms
Shi-Tomasi: 7.934 ms
```

```
RASTREAMENTO LUCAS-KANADE
------------------------------------------------------------
Frame 0001: vetores=200 | deslocamento=1.42 px | velocidade_aparente=42.58 px/s | LK=11.815 ms
Frame 0002: vetores=200 | deslocamento=1.41 px | velocidade_aparente=42.17 px/s | LK=12.544 ms
Frame 0003: vetores=200 | deslocamento=1.40 px | velocidade_aparente=42.07 px/s | LK=16.436 ms
Frame 0004: vetores=200 | deslocamento=1.51 px | velocidade_aparente=45.16 px/s | LK=12.170 ms
Frame 0005: vetores=200 | deslocamento=1.52 px | velocidade_aparente=45.44 px/s | LK=23.528 ms
Frame 0006: vetores=200 | deslocamento=1.56 px | velocidade_aparente=46.83 px/s | LK=15.285 ms
Frame 0007: vetores=200 | deslocamento=1.58 px | velocidade_aparente=47.21 px/s | LK=18.723 ms
Frame 0008: vetores=200 | deslocamento=1.63 px | velocidade_aparente=48.71 px/s | LK=16.893 ms
Frame 0009: vetores=200 | deslocamento=1.57 px | velocidade_aparente=47.12 px/s | LK=15.857 ms
Frame 0010: vetores=200 | deslocamento=1.65 px | velocidade_aparente=49.58 px/s | LK=17.682 ms
Frame 0011: vetores=200 | deslocamento=1.70 px | velocidade_aparente=50.85 px/s | LK=14.623 ms
Frame 0012: vetores=200 | deslocamento=1.73 px | velocidade_aparente=51.76 px/s | LK=15.097 ms
Frame 0013: vetores=200 | deslocamento=1.76 px | velocidade_aparente=52.63 px/s | LK=15.975 ms
Frame 0014: vetores=200 | deslocamento=1.77 px | velocidade_aparente=53.10 px/s | LK=13.028 ms
Frame 0015: vetores=200 | deslocamento=1.79 px | velocidade_aparente=53.70 px/s | LK=16.146 ms
Frame 0016: vetores=200 | deslocamento=1.86 px | velocidade_aparente=55.76 px/s | LK=11.600 ms
Frame 0017: vetores=200 | deslocamento=1.87 px | velocidade_aparente=56.09 px/s | LK=18.199 ms
Frame 0018: vetores=200 | deslocamento=1.99 px | velocidade_aparente=59.51 px/s | LK=20.096 ms
Frame 0019: vetores=200 | deslocamento=1.92 px | velocidade_aparente=57.52 px/s | LK=17.509 ms
```

```
Frame 0070: vetores=186 | deslocamento=5.38 px | velocidade_aparente=161.21 px/s | LK=21.736 ms
Frame 0071: vetores=186 | deslocamento=5.35 px | velocidade_aparente=160.35 px/s | LK=16.315 ms
Frame 0072: vetores=186 | deslocamento=6.09 px | velocidade_aparente=182.41 px/s | LK=12.171 ms
Frame 0073: vetores=185 | deslocamento=5.75 px | velocidade_aparente=172.33 px/s | LK=14.102 ms
Frame 0074: vetores=184 | deslocamento=5.54 px | velocidade_aparente=166.11 px/s | LK=15.474 ms
Frame 0075: vetores=183 | deslocamento=4.75 px | velocidade_aparente=142.50 px/s | LK=14.246 ms
Frame 0076: vetores=183 | deslocamento=5.29 px | velocidade_aparente=158.62 px/s | LK=13.146 ms
Frame 0077: vetores=183 | deslocamento=6.90 px | velocidade_aparente=206.85 px/s | LK=15.704 ms
Frame 0078: vetores=183 | deslocamento=5.22 px | velocidade_aparente=156.35 px/s | LK=12.938 ms

RESUMO DO FLUXO OPTICO
Frames processados: 78
Video salvo em: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex3\saidas\itemB\fluxo_optico.mp4

EXERCICIO FINALIZADO
Resultados em: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex3\saidas\itemB
```

**Justificativa:**
Neste exercício, utilizei dois métodos para identificar pontos de interesse nas imagens: Harris Corner Detector, com cv2.cornerHarris, e Shi-Tomasi, com cv2.goodFeaturesToTrack. Apliquei os dois métodos nas mesmas imagens para comparar a quantidade de pontos encontrados e o tempo que cada um levou para realizar a detecção. No Harris, selecionei os pontos com as respostas mais fortes, mantendo uma distância mínima entre eles. Já no Shi-Tomasi, defini um limite de 200 pontos e ajustei os parâmetros de qualidade e distância mínima. Depois, utilizei os pontos encontrados pelo Shi-Tomasi para testar o fluxo óptico Lucas-Kanade, usando a função cv2.calcOpticalFlowPyrLK. Com isso, foi possível acompanhar o movimento dos pontos entre os frames de um vídeo de trânsito. Para visualizar melhor esses movimentos, desenhei setas amarelas indicando a direção e o deslocamento dos pontos.
Também calculei o deslocamento médio em pixels e estimei a velocidade aparente em pixels por segundo, levando em consideração a quantidade de frames por segundo do vídeo.

**Análise do resultado:**
Os resultados mostraram algumas diferenças entre os dois detectores. O Harris encontrou menos pontos em quatro das cinco imagens, enquanto o Shi-Tomasi chegou ao limite de 200 pontos em quatro imagens e encontrou 182 na última. Observando as imagens, percebi que o Shi-Tomasi conseguiu distribuir melhor os pontos pela cena, identificando regiões como placas, veículos, árvores, construções e marcações da pista. Já o Harris destacou principalmente os cantos mais fortes, encontrando menos pontos em algumas imagens.
Na comparação do tempo de processamento, o Shi-Tomasi também foi mais rápido, com uma média de 7,934 ms, enquanto o Harris levou em média 12,588 ms.
Na parte do fluxo óptico, o Lucas-Kanade conseguiu acompanhar a maior parte dos pontos durante os 78 frames analisados. As setas desenhadas nas imagens mostraram os movimentos em diferentes direções e intensidades, principalmente em postes, semáforos, árvores e outros elementos da rua.
Também observei que o deslocamento médio dos pontos aumentou ao longo dos frames. Porém, isso não significa necessariamente que os obstáculos estavam acelerando, já que o movimento da câmera e a distância dos objetos também influenciam esse resultado.
Em um veículo autônomo, o fluxo óptico pode ajudar a perceber quais regiões estão se movimentando de maneira diferente do restante da imagem e acompanhar possíveis obstáculos. Com essas informações, é possível ter uma ideia da velocidade relativa aparente dos objetos.
Entretanto, para calcular velocidades reais, em metros por segundo, seria necessário considerar outros fatores, como a calibração da câmera, a distância dos objetos e o movimento do próprio veículo.

**Conclusão:**
Nos testes realizados, o Shi-Tomasi apresentou maior quantidade de pontos detectados e menor tempo médio de processamento, mostrando-se adequado para fornecer keypoints ao rastreamento.
O Lucas-Kanade conseguiu acompanhar os pontos entre frames consecutivos e representar visualmente seus deslocamentos por meio de vetores de movimento.
Apesar de não fornecer diretamente a velocidade real dos obstáculos, o fluxo óptico demonstrou ser uma técnica útil para analisar o movimento aparente da cena e auxiliar na percepção de veículos autônomos.

## Exercício 4

**Item A:**

**Comandos de instalação:**
pip install metadrive opencv-python numpy

**Comandos de execução:**
python .\Ex4\ex4_itemA.py

**Evidências de execução:**

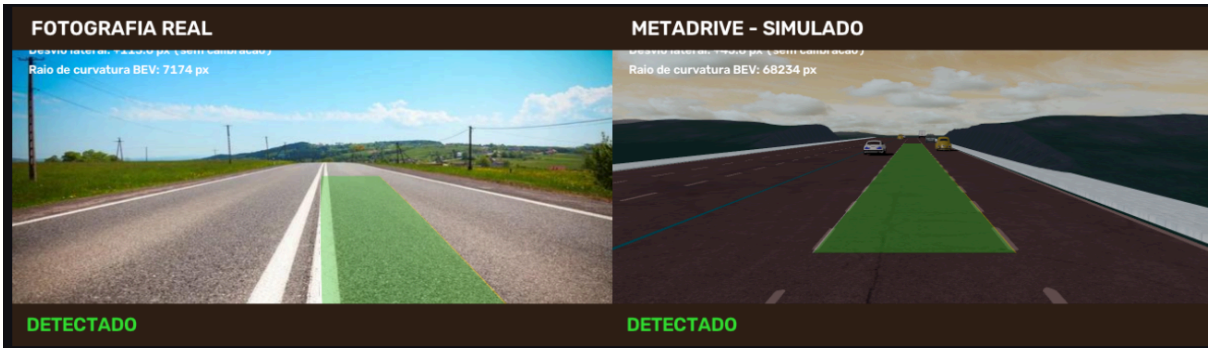
## Detecção de faixas durante a simulação




## Falha de detecção durante a simulação




## Comparação entre imagem real e simulada



**Terminal:**

```
=== EXERCICIO 4 - ITEM A ===
Iniciando MetaDrive...
Step 001/200 | Episodio 1 | Desvio: indisponivel | Velocidade: 0.2 km/h | Detector: 51.9 ms
Step 002/200 | Episodio 1 | Desvio: indisponivel | Velocidade: 0.5 km/h | Detector: 28.5 ms
Step 003/200 | Episodio 1 | Desvio: indisponivel | Velocidade: 0.8 km/h | Detector: 31.6 ms
Step 004/200 | Episodio 1 | Desvio: +45.8 px | Velocidade: 1.0 km/h | Detector: 50.0 ms
Step 005/200 | Episodio 1 | Desvio: +43.9 px | Velocidade: 1.3 km/h | Detector: 48.6 ms
Step 006/200 | Episodio 1 | Desvio: +49.4 px | Velocidade: 1.6 km/h | Detector: 54.9 ms
Step 007/200 | Episodio 1 | Desvio: +51.3 px | Velocidade: 1.8 km/h | Detector: 51.2 ms
Step 008/200 | Episodio 1 | Desvio: +50.1 px | Velocidade: 2.1 km/h | Detector: 51.9 ms
Step 009/200 | Episodio 1 | Desvio: +51.2 px | Velocidade: 2.3 km/h | Detector: 51.3 ms
Step 010/200 | Episodio 1 | Desvio: +48.3 px | Velocidade: 2.5 km/h | Detector: 48.5 ms
Step 011/200 | Episodio 1 | Desvio: +36.5 px | Velocidade: 2.8 km/h | Detector: 47.8 ms
Step 012/200 | Episodio 1 | Desvio: +26.4 px | Velocidade: 3.0 km/h | Detector: 71.2 ms
Step 013/200 | Episodio 1 | Desvio: +2.6 px | Velocidade: 3.2 km/h | Detector: 55.6 ms
Step 014/200 | Episodio 1 | Desvio: -5.5 px | Velocidade: 3.4 km/h | Detector: 57.0 ms
Step 015/200 | Episodio 1 | Desvio: -3.2 px | Velocidade: 3.7 km/h | Detector: 59.5 ms
Step 016/200 | Episodio 1 | Desvio: -3.8 px | Velocidade: 4.0 km/h | Detector: 89.2 ms
Step 017/200 | Episodio 1 | Desvio: -7.8 px | Velocidade: 4.2 km/h | Detector: 47.7 ms
Step 018/200 | Episodio 1 | Desvio: +0.2 px | Velocidade: 4.4 km/h | Detector: 56.0 ms
Step 019/200 | Episodio 1 | Desvio: +4.1 px | Velocidade: 4.7 km/h | Detector: 49.5 ms
Step 020/200 | Episodio 1 | Desvio: +26.1 px | Velocidade: 4.9 km/h | Detector: 53.5 ms
Step 021/200 | Episodio 1 | Desvio: +41.3 px | Velocidade: 5.2 km/h | Detector: 52.0 ms
Step 022/200 | Episodio 1 | Desvio: +43.9 px | Velocidade: 5.4 km/h | Detector: 49.1 ms
Step 023/200 | Episodio 1 | Desvio: +27.1 px | Velocidade: 5.7 km/h | Detector: 39.7 ms
Step 024/200 | Episodio 1 | Desvio: +15.7 px | Velocidade: 5.9 km/h | Detector: 41.3 ms
Step 025/200 | Episodio 1 | Desvio: +10.2 px | Velocidade: 6.1 km/h | Detector: 45.7 ms
Step 026/200 | Episodio 1 | Desvio: +8.2 px | Velocidade: 6.3 km/h | Detector: 40.8 ms
Step 027/200 | Episodio 1 | Desvio: +10.7 px | Velocidade: 6.6 km/h | Detector: 40.3 ms
Step 028/200 | Episodio 1 | Desvio: -12.6 px | Velocidade: 6.8 km/h | Detector: 41.2 ms
Step 029/200 | Episodio 1 | Desvio: -23.9 px | Velocidade: 7.1 km/h | Detector: 40.7 ms
Step 030/200 | Episodio 1 | Desvio: -13.2 px | Velocidade: 7.3 km/h | Detector: 41.0 ms
```

```
Step 198/200 | Episodio 3 | Desvio: indisponivel | Velocidade: 14.7 km/h | Detector: 67.9 ms
Step 199/200 | Episodio 3 | Desvio: indisponivel | Velocidade: 14.9 km/h | Detector: 29.7 ms
Step 200/200 | Episodio 3 | Desvio: indisponivel | Velocidade: 15.1 km/h | Detector: 35.9 ms

Comparacao salva: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex4\saidas\itemA\comparacao_real_simulada.png
Fotografia real: detectado=True
MetaDrive: detectado=True

=== RESULTADOS ===
Steps registrados: 200
Curvatura em pixels: estimativa no plano BEV, nao em metros.
Deteccoes validas: 85/200
CSV: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex4\saidas\itemA\metricas_200_steps.csv
Resultados: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex4\saidas\itemA
Validacao de 200 steps: OK
```

**Justificativa:**

Neste item, utilizei o detector de faixas que desenvolvi no TP1 dentro do simulador MetaDrive, para observar como ele funcionaria em um ambiente virtual.

Para isso, configurei uma câmera frontal usando use_render=True e image_observation=True. A cada step da simulação, o programa captura uma imagem e aplica o detector de faixas, que utiliza processamento de cores, Sobel, transformação de perspectiva BEV, transformada de Hough e ajuste polinomial para identificar as marcações da pista. Durante o teste, o veículo foi controlado por comandos aleatórios, com pequenas variações na direção e aceleração positiva, para que ele pudesse se movimentar pelo cenário. Também salvei os resultados em um arquivo CSV, registrando o desvio lateral, a curvatura estimada, a velocidade e as informações sobre as faixas detectadas.

**Análise do resultado:**
Durante a simulação, o detector conseguiu identificar as faixas em parte dos frames, destacando a região da pista em verde e apresentando as medidas calculadas sobre a imagem. Foram obtidas 85 detecções válidas em 200 steps. Nos demais frames, o detector não conseguiu identificar as duas delimitações necessárias. Isso ocorreu principalmente quando o veículo se deslocou lateralmente, fazendo com que as marcações deixassem as regiões de busca configuradas.
A comparação visual também mostrou que o algoritmo conseguiu detectar faixas tanto na fotografia real quanto na imagem simulada. Entretanto, foram observadas pequenas diferenças no alinhamento da região destacada, devido às perspectivas distintas das câmeras. O desvio lateral e o raio de curvatura foram registrados em unidades de imagem, pois não foi realizada uma calibração métrica da câmera. Dessa forma, esses valores representam estimativas geométricas no plano BEV, e não medidas físicas em metros.

**Conclusão:**
O programa realizou a captura dos frames, apresentou o desvio lateral em tempo real, registrou as métricas em CSV e gerou uma comparação visual entre imagens reais e simuladas. Apesar das falhas de detecção em algumas situações, os resultados demonstraram o funcionamento do detector no ambiente simulado e evidenciaram suas limitações diante das mudanças de posição do veículo e de perspectiva da câmera.

**Item B:**

**Comandos de instalação:**
python -m pip install ultralytics matplotlib

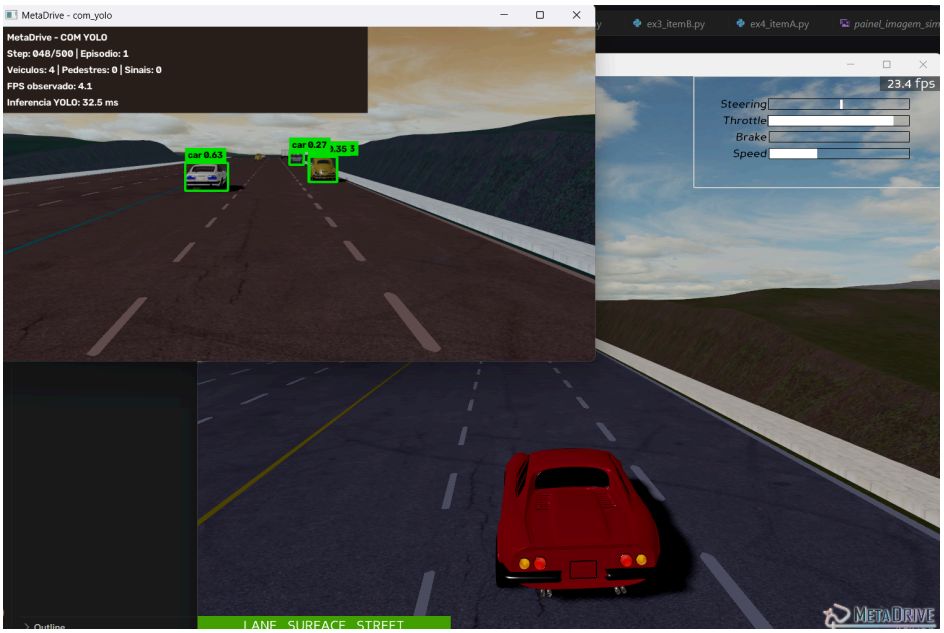
**Comandos de execução:**
python .\Ex4\ex4_itemB.py

**Evidências de execução:**

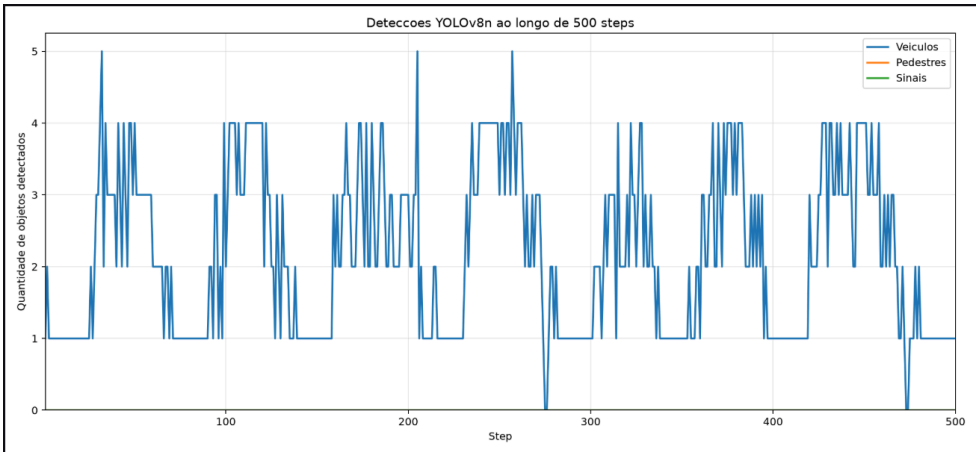
Simulação sem YOLO



## Simulação com YOLO




## Série temporal de detecções




## Terminal:

```
sem_yolo | Step 013/500 | FPS: 3.1 | Objetos: 0
sem_yolo | Step 014/500 | FPS: 3.1 | Objetos: 0
sem_yolo | Step 015/500 | FPS: 3.2 | Objetos: 0
sem_yolo | Step 016/500 | FPS: 3.3 | Objetos: 0
sem_yolo | Step 017/500 | FPS: 3.3 | Objetos: 0
sem_yolo | Step 018/500 | FPS: 3.4 | Objetos: 0
sem_yolo | Step 019/500 | FPS: 3.4 | Objetos: 0
sem_yolo | Step 020/500 | FPS: 3.5 | Objetos: 0
sem_yolo | Step 021/500 | FPS: 3.5 | Objetos: 0
sem_yolo | Step 022/500 | FPS: 3.5 | Objetos: 0
sem_yolo | Step 023/500 | FPS: 3.6 | Objetos: 0
sem_yolo | Step 024/500 | FPS: 3.6 | Objetos: 0
sem_yolo | Step 025/500 | FPS: 3.6 | Objetos: 0
sem_yolo | Step 026/500 | FPS: 3.7 | Objetos: 0
sem_yolo | Step 027/500 | FPS: 3.7 | Objetos: 0
sem_yolo | Step 028/500 | FPS: 3.7 | Objetos: 0
sem_yolo | Step 029/500 | FPS: 3.8 | Objetos: 0
sem_yolo | Step 030/500 | FPS: 3.8 | Objetos: 0
sem_yolo | Step 031/500 | FPS: 3.8 | Objetos: 0
```

```
=== INICIANDO COM_YOLO ===
com_yolo | Step 001/500 | FPS: 1.0 | Objetos: 1
com_yolo | Step 002/500 | FPS: 1.4 | Objetos: 2
com_yolo | Step 003/500 | FPS: 1.7 | Objetos: 1
com_yolo | Step 004/500 | FPS: 2.0 | Objetos: 1
com_yolo | Step 005/500 | FPS: 2.2 | Objetos: 1
com_yolo | Step 006/500 | FPS: 2.4 | Objetos: 1
com_yolo | Step 007/500 | FPS: 2.6 | Objetos: 1
com_yolo | Step 008/500 | FPS: 2.8 | Objetos: 1
com_yolo | Step 009/500 | FPS: 2.9 | Objetos: 1
com_yolo | Step 010/500 | FPS: 3.0 | Objetos: 1
com_yolo | Step 011/500 | FPS: 3.1 | Objetos: 1
com_yolo | Step 012/500 | FPS: 3.2 | Objetos: 1
com_yolo | Step 013/500 | FPS: 3.3 | Objetos: 1
com_yolo | Step 014/500 | FPS: 3.3 | Objetos: 1
com_yolo | Step 015/500 | FPS: 3.4 | Objetos: 1
```

```
com_yolo | Step 486/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 487/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 488/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 489/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 490/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 491/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 492/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 493/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 494/500 | FPS: 3.4 | Objetos: 1
com_yolo | Step 495/500 | FPS: 3.5 | Objetos: 1
com_yolo | Step 496/500 | FPS: 3.5 | Objetos: 1
com_yolo | Step 497/500 | FPS: 3.5 | Objetos: 1
com_yolo | Step 498/500 | FPS: 3.5 | Objetos: 1
com_yolo | Step 499/500 | FPS: 3.5 | Objetos: 1
com_yolo | Step 500/500 | FPS: 3.5 | Objetos: 1

==================================================
COMPARACAO DE DESEMPENHO
==================================================
FPS medio sem YOLO: 3.56
FPS medio com YOLO: 3.46
Degradacao de FPS: 2.89%
Tempo medio por inferencia YOLO: 34.00 ms
Inferencias realizadas: 500
Steps sem YOLO: 500
Steps com YOLO: 500

=== ARQUIVOS GERADOS ===
CSV: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex4\saidas\itemB\deteccoes_500_steps.csv
Grafico: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex4\saidas\itemB\serie_temporal_deteccoes.png
FPS: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex4\saidas\itemB\comparacao_fps.txt
Pasta de saida: C:\Users\scame\Documents\TP2-VeiculosAutonomosRoboticaMovel\Ex4\saidas\itemB

Validacao de 500 steps: OK
```

**Justificativa:**
Neste item, foi integrado o detector YOLOv8n ao MetaDrive para identificar objetos nas imagens capturadas pela câmera frontal do veículo.
O programa realiza a detecção de veículos, pedestres e sinais, desenhando caixas sobre os objetos identificados e mostrando a classe e a confiança de cada detecção.
Também foram realizadas duas execuções de 500 steps, uma sem YOLO e outra com YOLO, para comparar o FPS e verificar o impacto do detector no desempenho da simulação. Durante a execução com YOLO, a quantidade de objetos identificados por categoria foi registrada em um arquivo CSV para gerar o gráfico temporal.

**Análise do resultado:**
O YOLO conseguiu identificar os veículos durante a simulação, mostrando as caixas de detecção com os nomes dos objetos e os valores de confiança. No gráfico, percebi que a quantidade de veículos detectados mudou ao longo dos 500 steps, chegando a cinco em alguns momentos. Durante o teste, não foram identificados pedestres nem sinais de trânsito, pois não existiam no cenário. Na comparação de desempenho, o FPS médio foi de 3,56 sem YOLO e 3,46 com YOLO, uma queda de 2,89%. O tempo médio de cada inferência foi de 34 ms. Isso mostrou que o YOLO acrescentou processamento à simulação, mas não causou uma diferença muito grande no FPS.
Mesmo assim, o FPS ficou baixo nos dois testes, o que indica que a própria simulação e a captura das imagens também podem estar exigindo bastante processamento.
Para tentar melhorar esse resultado, uma opção seria executar o YOLO a cada dois ou três frames, reduzindo a quantidade de vezes que o detector precisa processar as imagens. Outra possibilidade seria diminuir a resolução das imagens utilizadas pelo YOLO. Porém, essas mudanças também podem ter desvantagens, como demorar mais para identificar novos objetos ou dificultar a detecção daqueles que estão mais distantes.

**Conclusão:**
A queda de 2,89% mostrou que o YOLO teve um impacto pequeno no desempenho dessa execução, embora o FPS geral tenha permanecido baixo. As estratégias de skip de frames e redução de resolução podem ajudar a melhorar o desempenho, mas precisam ser avaliadas considerando também a qualidade das detecções.
Considerando os resultados obtidos, eu escolheria inicialmente o skip de frames, executando o YOLO a cada dois frames. Essa estratégia reduziria a quantidade de inferências sem diminuir a resolução das imagens. Porém, seria necessário realizar novos testes para confirmar o ganho de FPS e verificar se a detecção dos objetos continuaria adequada.